

Tiny Shop Observability Platform

Overview

Tiny Shop Observability Platform is a cloud-native microservices application built to demonstrate modern observability practices using the three pillars of observability:

- **Logs**
- **Metrics**
- **Distributed Traces**

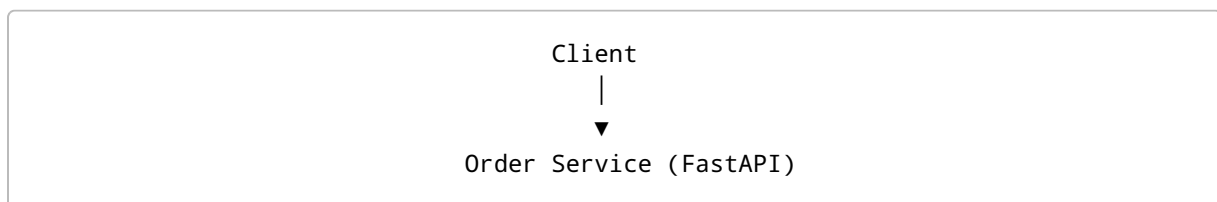
The project simulates a small e-commerce platform consisting of independent services that communicate over HTTP while generating telemetry that can be collected, analyzed, and correlated across multiple observability tools.

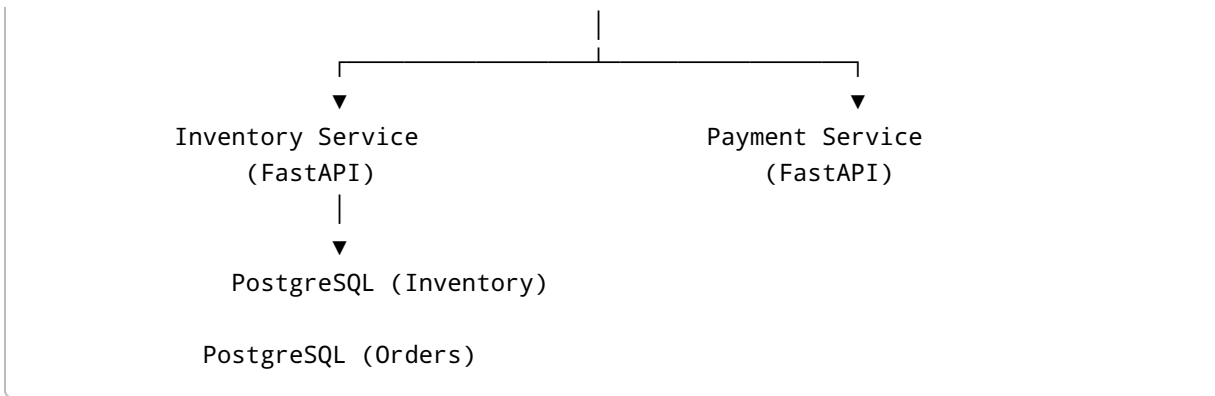
Objectives

The project was designed to learn and demonstrate:

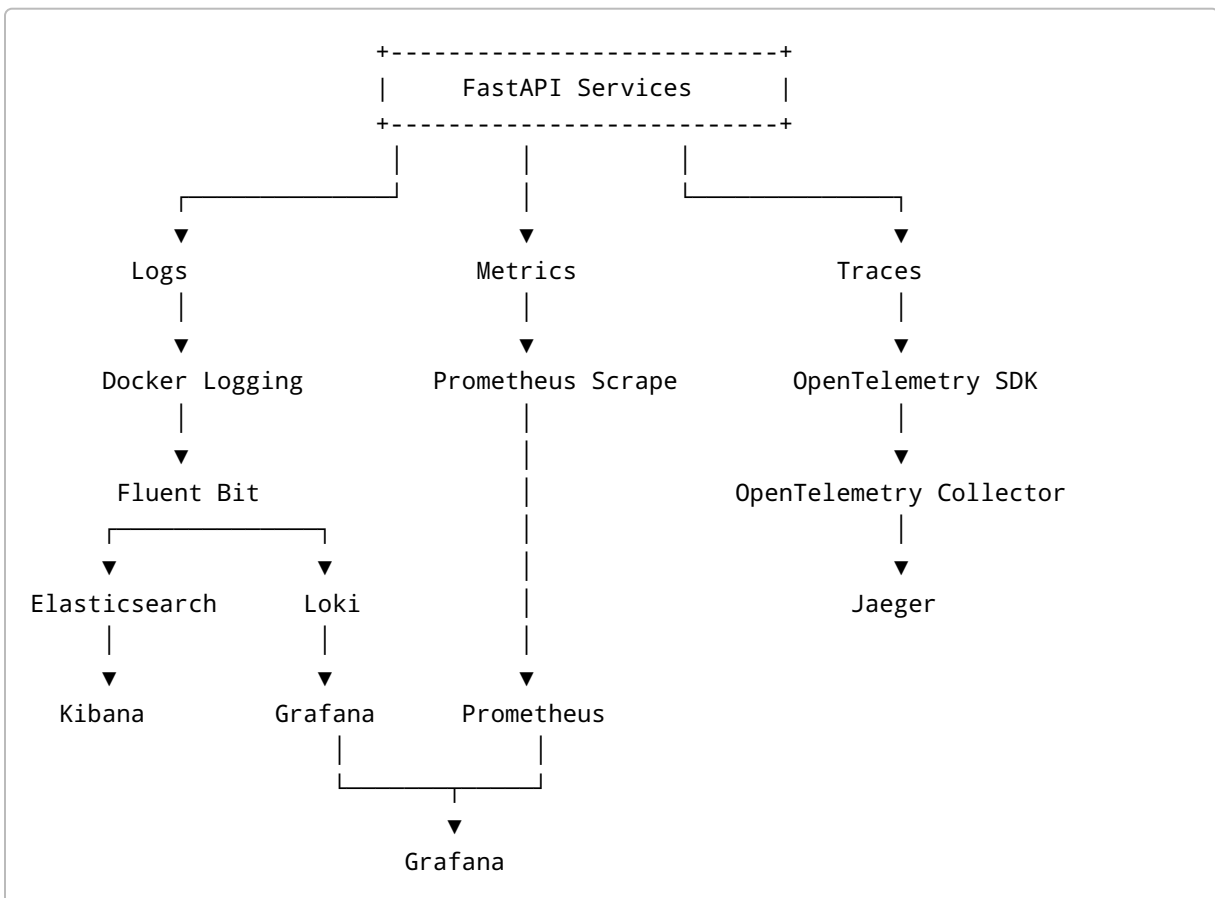
- Microservice architecture
 - Distributed tracing
 - Centralized logging
 - Metrics collection
 - Alerting
 - Incident investigation
 - Telemetry correlation
 - Fluent Bit log processing
 - OpenTelemetry instrumentation
 - Prometheus monitoring
 - Grafana dashboards
 - Elasticsearch log analytics
 - Loki log aggregation
 - Jaeger tracing
-

High-Level Architecture

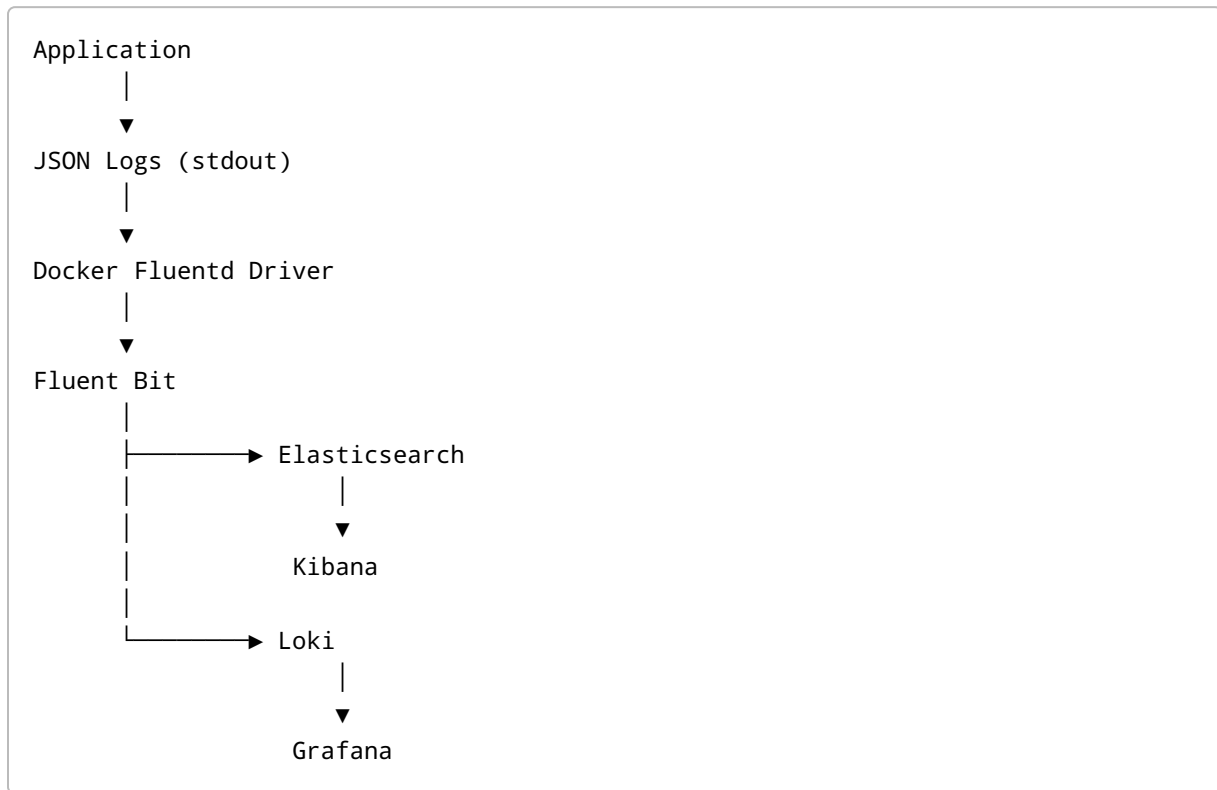




Observability Architecture



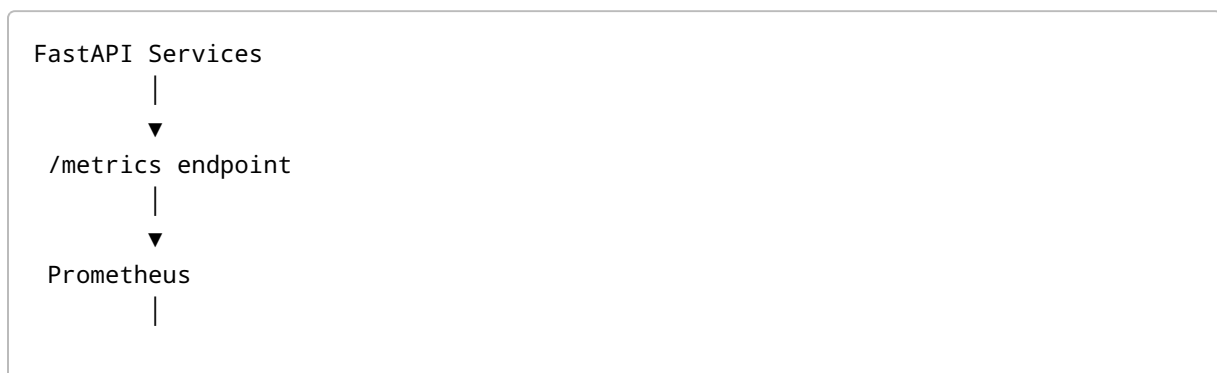
Logging Pipeline



Responsibilities:

- Applications produce structured JSON logs.
- Fluent Bit enriches and routes logs.
- Elasticsearch enables powerful full-text search.
- Loki provides log correlation with traces inside Grafana.

Metrics Pipeline

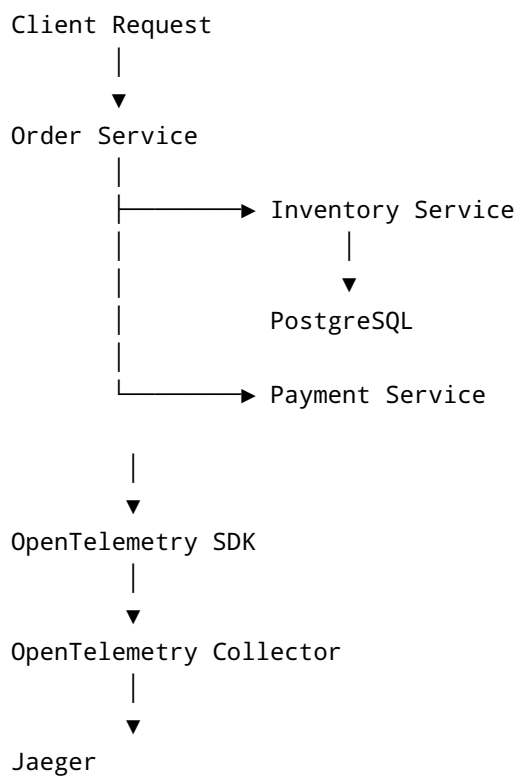


▼
Grafana

Collected metrics include:

- HTTP request count
- Request latency
- Error rate
- Order count
- Payment success/failure
- Inventory stock
- Reservation failures

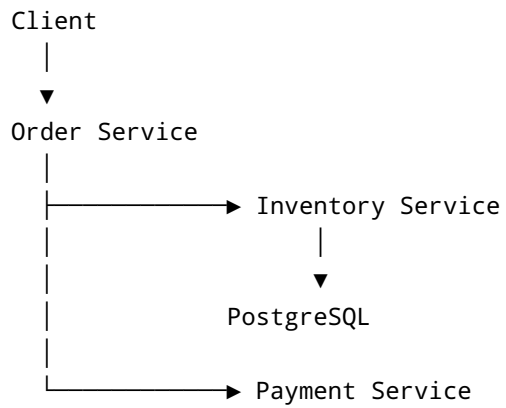
Distributed Tracing Pipeline



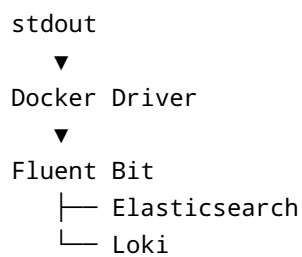
Every HTTP request generates a distributed trace that includes:

- Server spans
- Client spans
- Database spans
- Custom business spans

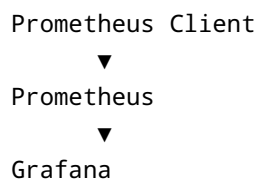
Complete Data Flow



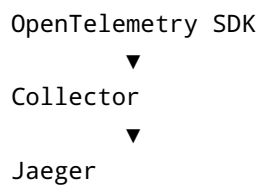
Logs



Metrics



Traces



Technology Stack

Component	Technology
Backend	Python 3.12
Framework	FastAPI
ORM	SQLAlchemy 2
Database	PostgreSQL
Logging	Fluent Bit
Log Storage	Elasticsearch
Log Visualization	Kibana
Log Correlation	Loki
Metrics	Prometheus
Dashboards	Grafana
Distributed Tracing	OpenTelemetry
Trace Storage	Jaeger
Containers	Docker Compose

Project Structure

```
tiny-shop/  
|  
├─ order-service/  
├─ inventory-service/  
├─ payment-service/  
|  
├─ fluent-bit/  
├─ prometheus/  
├─ grafana/  
├─ loki/  
├─ jaeger/  
├─ otel-collector/  
|  
├─ scripts/  
├─ dashboards/  
└─ alerting/
```

```
|  
└─ docker-compose.yml
```

Observability Journey

Stage 1

Application Development

- Three FastAPI services
 - PostgreSQL
 - REST APIs
-

Stage 2

Centralized Logging

- Structured JSON logs
 - Fluent Bit
 - Elasticsearch
 - Kibana
-

Stage 3

Distributed Tracing

- OpenTelemetry
 - Jaeger
 - Context propagation
 - Database spans
-

Stage 4

Metrics

- Prometheus
- Grafana
- Business metrics
- HTTP metrics

Stage 5

Dual Log Destinations

- Elasticsearch
- Loki
- Fluent Bit multiple outputs

Stage 6

Telemetry Correlation

- Metrics → Traces
- Traces → Logs
- Grafana unified investigation

Stage 7

Alerting

- Prometheus Alert Rules
- Alertmanager
- Latency alerts
- Payment failure alerts
- Low inventory alerts
- Service availability alerts

Example Incident Investigation

Scenario

Customers report that checkout is slow.

Step 1

Grafana dashboard shows:

- High p95 latency
- Increased payment failures

↓

Step 2

Open Jaeger.

Find a slow trace.

The trace shows:

- Inventory: 20 ms
- Payment: 3100 ms

↓

Step 3

Open logs in Grafana.

Search using:

```
trace_id=<trace-id>
```

↓

Step 4

The logs reveal:

```
Payment provider timeout
```

↓

Root Cause

The external payment service exceeded the configured timeout.

Monitoring Dashboards

The project includes dashboards for:

- Application Health

- HTTP Requests
 - Request Latency
 - Error Rate
 - Order Statistics
 - Payment Statistics
 - Inventory Levels
 - Fluent Bit Health
 - Prometheus Targets
-

Alerts

Configured alerts include:

- High HTTP latency
 - High error rate
 - Payment failure threshold exceeded
 - Low inventory
 - Service unavailable
-

Key Learnings

This project demonstrates:

- Designing a microservice architecture
 - Building resilient REST services
 - Instrumenting applications with OpenTelemetry
 - Collecting and routing logs using Fluent Bit
 - Managing logs in Elasticsearch and Loki
 - Creating metrics with Prometheus
 - Visualizing telemetry in Grafana
 - Building distributed traces in Jaeger
 - Creating actionable alerts
 - Performing end-to-end incident investigations using logs, metrics, and traces
-

Future Improvements

Potential enhancements include:

- Kubernetes deployment
- Helm charts
- OpenTelemetry Metrics SDK

- Distributed message queues (Kafka/RabbitMQ)
- Authentication with Keycloak
- CI/CD with GitHub Actions
- Load testing using k6
- Horizontal scaling
- Service mesh integration with Istio
- Production-grade alert routing (Slack, Microsoft Teams, PagerDuty, email)